

EXERCICE FRONT-END — REACT

LaunchPad — Le frontend

Donner une interface au centre de contrôle des lancements

Service Web (25604) • Application front-end
React (Vite) · useState · useEffect · axios · useContext · JWT

Rappel. Vous avez déjà construit **tout le backend** de LaunchPad : une API REST (Express) qui va chercher les vrais prochains lancements sur *The Space Devs*, les stocke sur Neon (Prisma), et gère l'authentification (JWT) ainsi que les suivis et commentaires. **Mais il n'a pas de visage** : on l'interrogeait avec Postman.

Votre mission maintenant : construire le **frontend React complet** qui consomme cette API — un vrai tableau de bord « Mission Control » utilisable dans le navigateur.

Objectifs pédagogiques

Cet exercice mobilise **tout** ce qui a été vu dans les séances React (S7 à S9) :

- Application **React (Vite)** organisée en **composants + props**.
- **useState** et **formulaires contrôlés**.
- **useEffect** + **axios** pour consommer **votre API**.
- **Listes** (`.map` + `key`), **événements**, **affichage conditionnel**.
- États **chargement** / **erreur** / **succès**.
- **Authentification côté client** : login, token (`localStorage`), header `Authorization`, **useContext**.
- Routes/actions **protégées** et réservées à un **rôle** (ADMIN).

Modalités

- **Durée** : 4 à 6 heures (idéalement sur plusieurs séances). **Format** : individuel ou en binôme.
- **Frontend uniquement** : vous *réutilisez* votre backend LaunchPad existant (aucune nouvelle route à créer, sauf bonus).
- Le backend doit **tourner** (port 3000) et **autoriser le front** (voir Prérequis).
- Détails (commandes, routes de l'API, aide-mémoire) : voir l'**Annexe**.

Prérequis CORS. Votre backend LaunchPad doit accepter les appels du front (port 5173). Si ce n'est pas déjà fait, activez `cors` côté backend (`npm install cors` puis `app.use(cors())` avant les routes). Sinon, le navigateur bloquera toutes les requêtes.

Consigne. Cet énoncé décrit **ce qu'il faut construire**, pas *comment* l'écrire. À vous de coder. L'**Annexe** ne donne que des *rappels* (commandes, routes de l'API, signatures) — aucune solution.

Mission 0 — Mise en place

[5 pts]

À faire

- Créer une app React + TypeScript avec **Vite**, installer **axios**.
- Créer une **instance axios** centralisée (l'URL de base de votre API), que vous réutiliserez partout.
- Vérifier que le backend répond (une requête simple sur la liste des lancements).

Mission 1 — Le mur des lancements

[20 pts]

L'écran d'accueil : la liste des prochains décollages.

À faire

- Au chargement, récupérer les lancements (**useEffect** + **axios**) et les afficher en **cartes** : nom, agence, fusée, lieu, date, statut, et l'**image** si disponible.
- Gérer les **3 états** : chargement, erreur, succès.
- Ajouter un **filtre par statut** (ex. À venir) et la **pagination** (page suivante / précédente), en passant les paramètres à l'API.
- Générer les cartes avec **.map()** + une **key** unique.

La liste des lancements est **publique** : pas besoin d'être connecté pour la voir. Le filtre et la pagination se font via la *query string* (voir Annexe).

Mission 2 — Détail d'un lancement & compte à rebours

[15 pts]

À faire

- Cliquer sur une carte ouvre le **détail** d'un lancement (récupéré par son **id**) : toutes ses infos + sa mission.
- Afficher un **compte à rebours** jusqu'à la date de décollage (jours / heures / minutes restants).
- Afficher la liste des **commentaires** de ce lancement (route publique).

Pour un compte à rebours qui *avance* en direct, pensez à un minuteur dans un **useEffect** — et à bien le **nettoyer** (fonction de retour). La navigation entre pages peut se faire à la main (un state « page courante ») ou avec **react-router** (bonus).

Mission 3 — Authentification

[25 pts]

À faire

- Un formulaire d'**inscription** et un formulaire de **connexion** (formulaires contrôlés) qui appellent vos routes `/auth/register` et `/auth/login`.
- À la connexion, **stocker le token** reçu et le **renvoyer automatiquement** dans le header `Authorization` à chaque requête protégée.
- Un **contexte d'authentification** (`useContext`) exposant l'état connecté, la connexion et la **déconnexion**.
- **Affichage conditionnel** : « Se connecter » vs « Se déconnecter », et rester connecté après un **rechargement** de la page.

Rappel de sécurité : cacher un bouton n'est **pas** une protection. La vraie sécurité est **côté backend** (le JWT que vous avez déjà). L'affichage conditionnel est du confort d'interface.

Mission 4 — Ma liste de suivi**[15 pts]****À faire (réservé aux utilisateurs connectés)**

- Un bouton « **Suivre** » / « **Ne plus suivre** » sur un lancement (crée / supprime un suivi).
- Une page/section « **Mes suivis** » qui affiche uniquement les lancements suivis par l'utilisateur connecté.
- Ces actions ne sont possibles que **connecté** (sinon, inviter à se connecter).

Mission 5 — Commenter**[10 pts]****À faire**

- Sur la page de détail, un **formulaire** pour ajouter un commentaire (connecté uniquement).
- Après envoi, **rafraîchir** la liste des commentaires.
- Permettre de **supprimer** un commentaire (le sien).

Mission 6 — Espace Admin**[5 pts]****À faire (visible seulement si rôle ADMIN)**

- Un bouton « **Importer les prochains lancements** » qui appelle la route d'import (Axios côté backend → The Space Devs) puis rafraîchit la liste.
- (Optionnel) **Supprimer** un lancement du catalogue.
- Ces actions n'apparaissent que pour un compte **ADMIN**.

Pour connaître le rôle côté front : soit vous appelez `/auth/me`, soit vous lisez le rôle contenu dans le token. Rappel : pour *obtenir* un compte ADMIN, on promeut un utilisateur dans Prisma Studio (côté backend).

Mission 7 — Finitions

[5 pts]

À faire

- Découper en **composants** réutilisables (carte de lancement, badge de statut, compte à rebours...).
- Un **style** soigné (thème « Mission Control » sombre, par exemple).
- Messages clairs partout (chargement, vide, erreurs).

Objectifs bonus

- **react-router** : de vraies pages (/, /lancement/:id, /login, /mes-suivis) avec une **route protégée**.
- **Déconnexion automatique** si une requête renvoie 401 (intercepteur de réponse axios).
- **Recherche / tri** des lancements (par agence, par date).
- **Rappel visuel** : mettre en évidence les lancements suivis avec l'option « rappel ».
- **Badges de statut** colorés (À venir, Réussi, Échec, Reporté).

Barème (sur 100)

Critère	Points
Mission 0 — Projet Vite + instance axios	5
Mission 1 — Liste + filtres + pagination	20
Mission 2 — Détail + compte à rebours + commentaires	15
Mission 3 — Authentification (login, token, contexte)	25
Mission 4 — Ma liste de suivi	15
Mission 5 — Commenter	10
Mission 6 — Espace Admin (import, rôle)	5
Mission 7 — Composants & finitions	5
Total	100
Objectifs bonus (au mérite, plafonné)	+10

Livrables

- Le **dépôt Git** du frontend (sans `node_modules`).
- Un **README** : comment lancer le front ET le back, et les fonctionnalités réalisées.
- Les **noms** des membres.

ANNEXE — Aide-mémoire (rappels)

Rappels de commandes, routes de votre API et signatures. Aucune solution.

A.1 — Créer l'application

```
npm create vite@latest launchpad-front -- --template react-ts
cd launchpad-front
npm install
npm install axios
npm run dev          # -> http://localhost:5173
```

A.2 — Activer CORS (côté backend LaunchPad)

```
npm install cors
# src/server.ts :
import cors from "cors"
app.use(cors())          // AVANT les routes
```

A.3 — Les routes de VOTRE backend (rappel)

Méth.	Route	Accès
POST	/auth/register	public
POST	/auth/login	public (renvoie un token)
GET	/auth/me	connecté
GET	/lancements?statut=A_VENIR&page=1&limit=10	public
GET	/lancements/:id	public
POST	/lancements/importer	connecté (import Axios)
DELETE	/lancements/:id	ADMIN
GET	/mes-suivis	connecté
POST	/mes-suivis	connecté (corps : lancementId)
DELETE	/mes-suivis/:id	propriétaire
GET	/lancements/:id/commentaires	public
POST	/lancements/:id/commentaires	connecté (corps : contenu)
DELETE	/commentaires/:id	auteur / ADMIN

A.4 — Aide-mémoire React (signatures)

```
const [valeur, setValeur] = useState(initiale)

useEffect(() => {
  /* effet : appel axios, minuteur... */
  return () => { /* nettoyage (ex. clearInterval) */ }
}, [dependances])          // [] une fois ; [x] quand x change

axios.get(url, { params, headers })          // -> res.data
axios.post(url, corps, { headers })
tableau.map((el) => <Composant key={...} />)

// auth cote client
localStorage.setItem("token", data.token)
config.headers.Authorization = `Bearer ${token}` // intercepteur

// contexte
createContext(...) ; <Provider value={...}> ; useContext(...)
```

A.5 — Repères

- **Compte à rebours** : `new Date(dateLancement).getTime() - Date.now()` donne les millisecondes restantes ; convertir en jours/heures/minutes. Rafraîchir avec un `setInterval` dans un `useEffect` (et le nettoyer).
- **Dates** : `new Date(chaine).toLocaleString('fr-CA')` pour un affichage lisible.
- **Instance axios** : centraliser l'URL de base et l'ajout du token (intercepteur) évite de se répéter.
- **Rôle ADMIN** : via `/auth/me` ou le rôle du token ; les actions admin ne s'affichent que dans ce cas.
- Toujours une `key` unique dans un `.map()` ; `e.preventDefault()` sur les formulaires.

Prêt pour le décollage — donnez enfin un visage à votre Mission Control !